Tripvault

Travel Memory & Exploration Tracker

Week 1 – Project Setup & User Authentication

A Project Report

NAME : KARTHIK R

Submitted as part of

Tripvault Virtual Internship Program

Powered by CodGen

Technology Stack: React.js (Vite) • Node.js • Express.js • MongoDB Atlas • JWT • bcrypt.js

1. Abstract

TripVault is a full-stack web application conceived to help travellers capture, organise, and revisit their journeys in a single, secure digital space. In an era where travel experiences are frequently scattered across scattered notes, disconnected photo albums, and generic social media posts, TripVault aims to provide a purpose-built platform where users can log destinations, preserve memories, and eventually visualise their travel history through analytics and interactive maps. This report documents the first milestone of the TripVault Virtual Internship Program powered by CodGen, focusing on the foundational phase of the project: environment setup and the implementation of a secure user authentication system.

The Week 1 deliverable establishes the technical backbone of the application. On the frontend, a React.js application scaffolded with Vite provides a fast, modern development environment with hot module replacement and optimised builds. The backend is built on Node.js using the Express.js framework, exposing a RESTful API layer that communicates with a cloud-hosted MongoDB Atlas database. Security is a central concern from the earliest stage of development: user passwords are never stored in plain text but are hashed using bcrypt.js, and authenticated sessions are managed statelessly using JSON Web Tokens (JWT). This approach ensures that TripVault's authentication layer follows industry-accepted best practices for web application security.

This report walks through the problem that TripVault addresses, the objectives and scope of the project, a comparison between existing and proposed systems, functional and non-functional requirements, the system architecture, database design, API specification, authentication workflow, folder structure, and the implementation details completed during Week 1. It further documents the algorithms used for registration and login, presents flowcharts of the authentication process, and includes placeholders for the screenshots that evidence a working implementation. The report concludes with the results achieved, the advantages and limitations of the current build, and a roadmap of future enhancements such as an interactive world map, travel analytics dashboard, and AI-based travel recommendations.

Overall, this document serves as both a technical record of the work completed and a professional demonstration of software engineering practices — including modular project structure, secure authentication design, and RESTful API development — applied to a real-world, production-style application.

2. Table of Contents

27. Appendix

- Installation Commands
- Environment Variables (.env)
- Sample JSON Request (Register)
- Sample JSON Response (Register)
- Sample JSON Request (Login)
- Sample JSON Response (Login)

3. List of Figures

4. List of Tables

5. Introduction

What is Tripvault?

Tripvault is a travel memory and exploration tracking platform that allows users to create an account, securely log in, and (in subsequent milestones) record trips, upload photographs, tag visited locations, and review a personalised timeline of their journeys. It is designed as a single-page application backed by a RESTful API, following modern full-stack development conventions.

Why is it Needed?

Travellers today generate a large volume of trip-related data — photographs, notes, itineraries, and expenses — but this data is rarely consolidated in one place. Existing tools either focus narrowly on photo storage, itinerary planning, or expense tracking, but not on all three combined with a personal, chronological travel narrative. Tripvault fills this gap by offering a unified space purpose-built for memory-keeping and exploration analytics.

Problems with Traditional Travel Journals

- Physical journals and scattered photo folders are prone to loss and damage.
- Generic social media platforms are optimised for public sharing, not private reflection or structured data.
- Existing note-taking apps lack travel-specific structures such as destination tagging, country tracking, or map visualisation.
- No single platform combines authentication-secured personal storage with analytics on travel history.

Motivation

The motivation behind Tripvault stems from the observation that travel memories are emotionally valuable but technically under-served. By combining a secure, modern web stack with a travel-centric data model, Tripvault aims to give users a durable, private, and visually engaging home for their travel history — while giving the development team a realistic, production-style project to practise full-stack engineering, authentication design, and cloud database integration.

Objectives

The Week 1 phase focuses specifically on establishing the technical foundation — project scaffolding, backend server configuration, database connectivity, and a secure authentication system — upon which all future Tripvault features will be built.

6. Problem Statement

There is currently no lightweight, secure, and travel-focused platform that allows individuals to register an account, safely store their credentials, and maintain an evolving digital record of their travel experiences. Users are forced to rely on a patchwork of unrelated tools — cloud photo storage, spreadsheets, and social media — none of which are designed around the specific needs of a traveller who wants to track destinations, revisit memories, and analyse their journeys over time. Tripvault addresses this problem by providing a dedicated, secure, full-stack application, beginning with a robust user authentication system as its foundation.

7. Objectives

- To design and implement a secure user registration and login system using JWT-based authentication.
- To ensure user passwords are never stored in plain text by applying bcrypt.js hashing.
- To establish a scalable backend architecture using Node.js and Express.js.
- To integrate a cloud-hosted MongoDB Atlas database for persistent data storage.
- To build a responsive React.js (Vite) frontend capable of consuming the authentication API.
- To implement protected routes that are only accessible to authenticated users.
- To establish a clean, modular folder structure that supports future feature expansion.
- To document the system architecture, database design, and API endpoints for maintainability.
- To lay the groundwork for future travel-tracking features such as maps, galleries, and analytics.

8. Scope of the Project

The scope of Week 1 of the Tripvault project is limited to establishing the project's technical foundation and implementing a complete, secure user authentication flow. This includes setting up the client and server codebases, configuring the MongoDB Atlas database, designing the User schema, and building the registration, login, and protected-route API endpoints. Frontend scope is limited to the pages and components necessary to interact with these authentication endpoints, including a registration form, a login form, and a basic protected dashboard page. Features such as trip logging, photo galleries, maps, and analytics — while part of the broader Tripvault vision — are outside the scope of this milestone and are addressed under Future Enhancements.

9. Existing System

Currently, travellers rely on a combination of generic tools to manage their travel memories: cloud photo storage services for images, spreadsheet applications for basic itinerary or expense tracking, and social media platforms for sharing highlights publicly. None of these tools were purpose-built for private, structured travel record-keeping.

Limitations of the Existing System

- Data is fragmented across multiple unrelated platforms with no unified view.
- No dedicated authentication or privacy model tailored to personal travel data.
- Lack of structured tagging for destinations, dates, and trip categories.
- No analytics or visualisation of travel history (e.g., countries visited, distance travelled).
- Social platforms prioritise public engagement over private, long-term memory-keeping.

10. Proposed System

Tripvault proposes a unified, secure, full-stack platform purpose-built for travel memory management. The system begins with a robust authentication layer, ensuring that every user's data is private and accessible only to them, and is designed to scale toward richer features such as trip timelines, interactive maps, and analytics dashboards in later milestones.

Advantages of the Proposed System

- Centralised, secure storage of user accounts and (in future) travel data.
- Industry-standard security practices: password hashing with bcrypt.js and stateless authentication with JWT.
- A modular, scalable Node.js/Express.js backend that can grow with new features.
- A modern, responsive React.js frontend built with Vite for fast development and performance.
- Cloud-hosted MongoDB Atlas database ensuring availability and easy scalability.
- Clear separation of concerns via a well-organised folder structure, easing long-term maintenance.

11. Functional Requirements

The functional requirements describe the specific behaviours the Tripvault system must support during the Week 1 milestone.

Requirement	Description
User Registration	Allow new users to create an account using name, email, and password.
User Login	Authenticate existing users using email and password credentials.
Password Encryption	Hash user passwords using bcrypt.js before storing them in the database.
JWT Authentication	Issue a signed JSON Web Token upon successful login for session management.
Protected Routes	Restrict access to certain API endpoints to requests bearing a valid JWT.
User Profile	Allow an authenticated user to retrieve their own profile information.
MongoDB Storage	Persist user records reliably in a MongoDB Atlas collection.

Table 5: Functional Requirements Summary

12. Non-Functional Requirements

Performance

API responses for authentication operations should complete within an acceptable time frame (typically under 300ms under normal load) to ensure a responsive user experience.

Reliability

The system should consistently persist and retrieve user data without corruption, leveraging MongoDB Atlas's replication and automated backup capabilities.

Security

All sensitive data, particularly passwords, must be encrypted at rest using bcrypt.js, and all authenticated communication must be validated via JWT signature verification.

Scalability

The backend architecture should support horizontal scaling as user load grows, aided by the stateless nature of JWT authentication.

Maintainability

The codebase follows a modular MVC-inspired structure (models, routes, controllers, middleware) to simplify debugging, testing, and future feature additions.

Availability

Cloud hosting of the database (MongoDB Atlas) ensures high availability, minimising downtime for end users.

13. Software Requirements

Software	Purpose / Version
Operating System	Windows 10/11, macOS, or Linux (development-environment agnostic)
Node.js	v18 LTS or later — JavaScript runtime for the backend server
Express.js	v4.x — Web application framework for building the REST API
React.js (Vite)	v18.x — Frontend library bundled using the Vite build tool
MongoDB Atlas	Cloud-hosted NoSQL database (free-tier M0 cluster)
Visual Studio Code	Primary source-code editor with extensions for JS/React
Git	v2.x — Distributed version control system
GitHub	Remote repository hosting for source control and collaboration
Postman	API testing and documentation tool

Table 1: Software Requirements

14. System Architecture

Tripvault follows a layered client-server architecture. The React frontend communicates with the Express backend over HTTP using Axios, and the backend interfaces with MongoDB Atlas for persistent storage. An authentication middleware layer sits between incoming requests and protected controllers, validating JWTs before allowing access to secured resources.

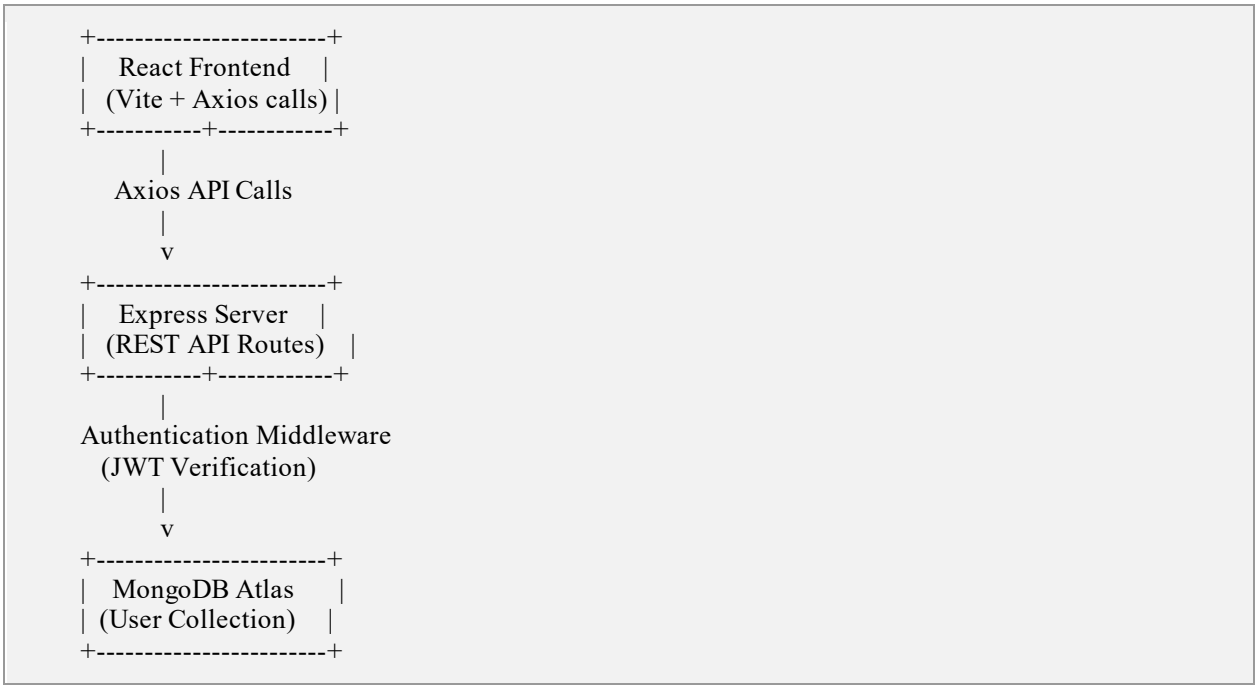


Figure 10: System Architecture Diagram

Layer-by-Layer Explanation

React Frontend

The presentation layer, built with React.js and bundled using Vite, renders the registration and login forms and manages client-side state such as form inputs and authentication status.

Axios API Calls

Axios is used as the HTTP client to send requests from the React frontend to the Express backend, handling JSON serialisation, headers (including the Authorization bearer token), and error responses.

Express Server

The Express.js application defines the REST API routes (/api/auth/register, /api/auth/login, /api/auth/me) and delegates request handling to controller functions.

Authentication Middleware

A custom middleware function intercepts requests to protected routes, extracts the JWT from the Authorization header, verifies its signature and expiry, and attaches the decoded user identity to the request object before passing control to the next handler.

MongoDB Atlas

The cloud-hosted database layer stores the User collection persistently, accessed through Mongoose object models that define schema validation and provide a convenient query interface.

Component Screenshots

The following screenshots, captured from the working Tripvault implementation, illustrate the three core components of the stack in practice: the React frontend, the Node.js/Express backend, and the MongoDB Atlas database.

Frontend

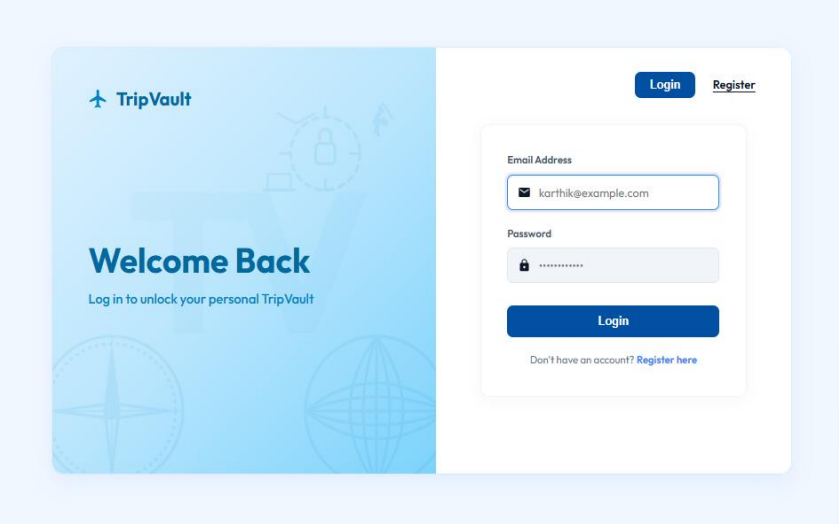


Figure 15: Tripvault Frontend — Login Page

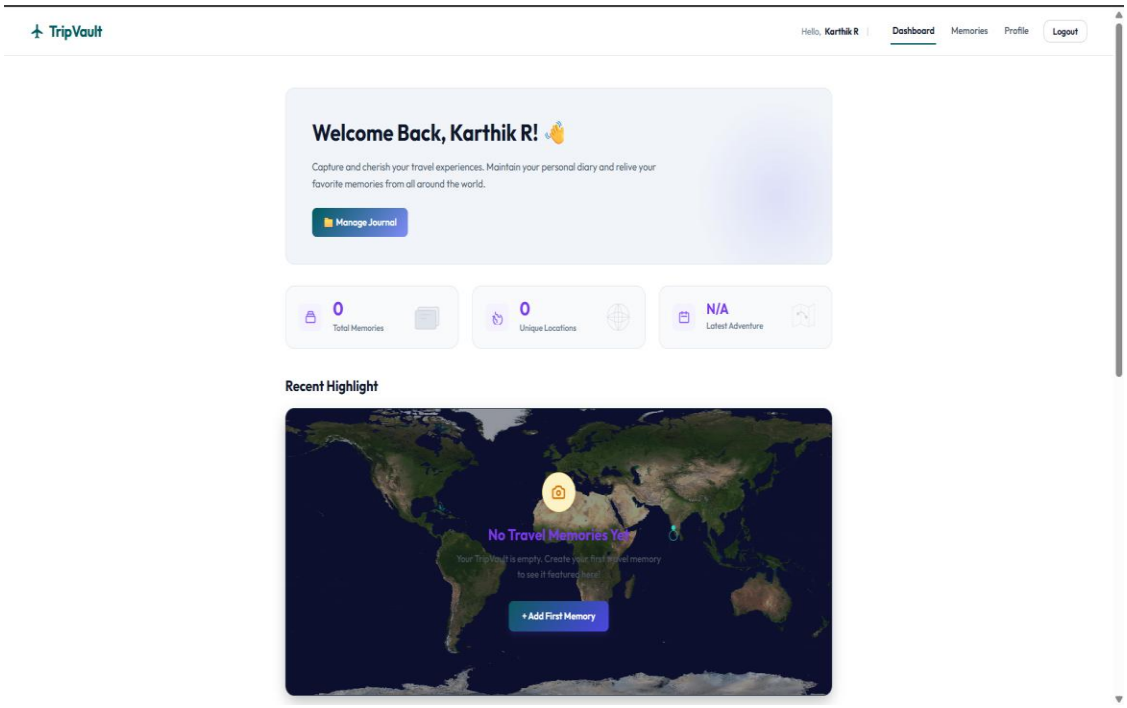


Figure 16: Tripvault Frontend — Dashboard

Backend

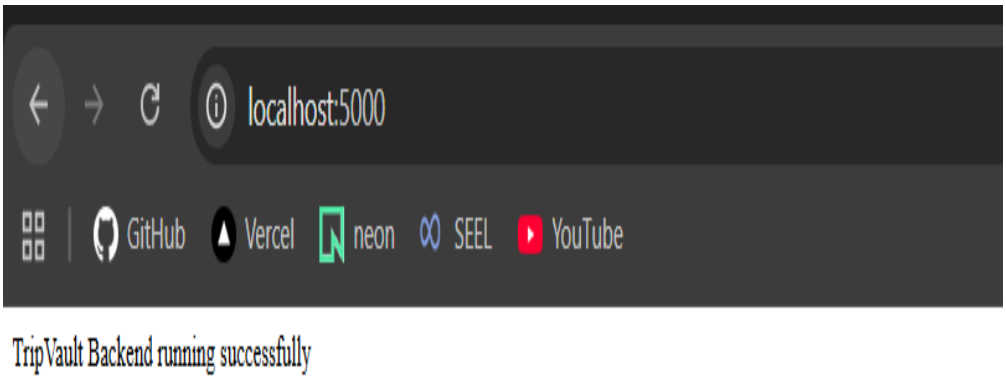


Figure 17: Tripvault Backend — Server Output

Database

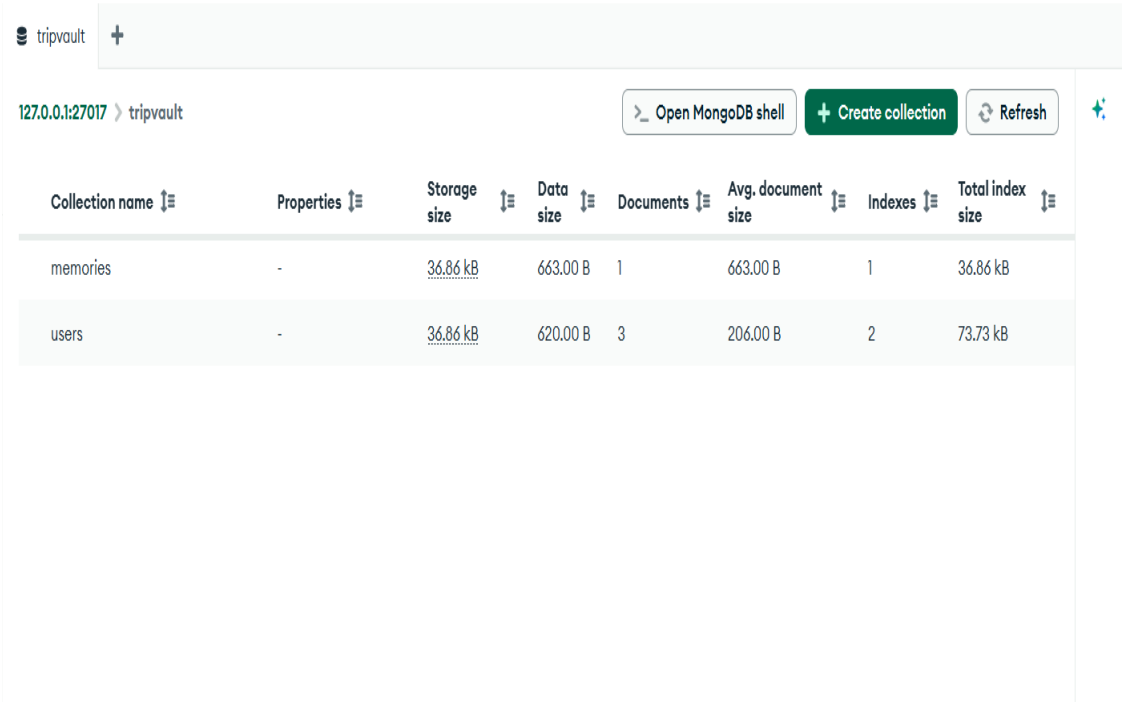


Figure 18: MongoDB Atlas — Cluster Overview

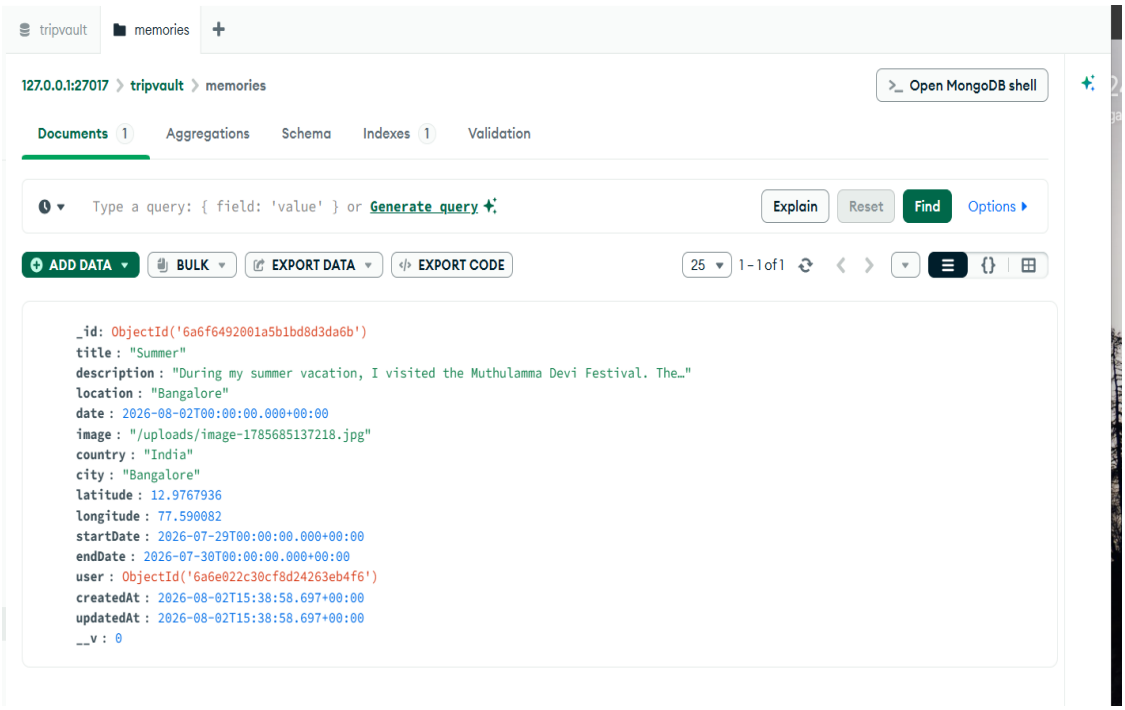


Figure 19: MongoDB Atlas — Database Collections

15. Database Design

The Week 1 milestone introduces a single core collection — the User collection — which stores account credentials and profile metadata for each registered user.

Field	Type	Description
_id	ObjectId	MongoDB-generated unique identifier for each user document.
name	String	Full name of the user, provided at registration.
email	String	Unique email address used as the login identifier.
password	String	bcrypt-hashed password; never stored in plain text.
createdAt	Date	Timestamp automatically recorded when the account is created.

Table 3: User Collection Schema

Field Explanation

- _id: Automatically generated by MongoDB as the primary key for each document.
- name: Captures the user's display name for personalising the interface.
- email: Enforced as unique at the schema level to prevent duplicate accounts.
- password: Stored only after being hashed with bcrypt.js, using a configured salt round count.
- createdAt: Enables auditing and future features such as account-age-based analytics.

16. API Endpoints

The following RESTful endpoints comprise the Week 1 authentication API surface.

Method	Endpoint	Description
POST	/api/auth/register	Registers a new user account after validating input and hashing the password.
POST	/api/auth/login	Authenticates a user's credentials and returns a signed JWT on success.
GET	/api/auth/me	Returns the authenticated user's profile; requires a valid JWT (protected route).

Table 4: API Endpoints

17. Authentication Workflow

The authentication workflow spans registration and login, culminating in secure access to protected resources. Each step is described below, followed by a visual representation of the flow.

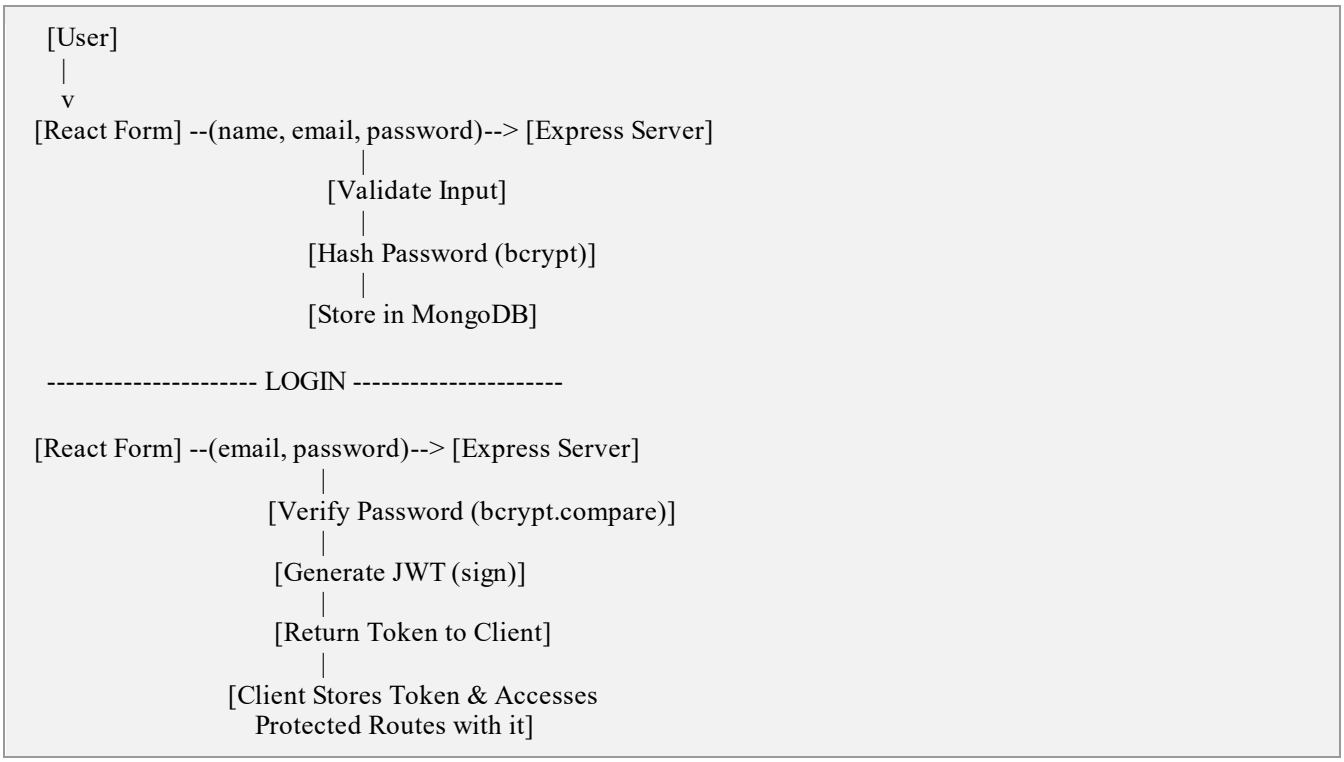


Figure 11: Authentication Workflow Diagram

Step-by-Step Explanation

1. User submits the registration form (name, email, password) from the React client.
2. The Express server receives the request and validates the input fields (presence, format, and email uniqueness).
3. On successful validation, the password is hashed using bcrypt.js with a configured number of salt rounds.
4. The hashed password, along with the user's name and email, is stored as a new document in the MongoDB User collection.
5. For login, the user submits their email and password through the React login form.
6. The server retrieves the matching user record by email and uses bcrypt.compare() to verify the submitted password against the stored hash.
7. If verification succeeds, the server generates a signed JWT containing the user's ID and an expiry claim.

8. The token is returned to the client, which stores it (e.g., in memory or local state) and attaches it as a Bearer token on subsequent requests.
9. Requests to protected routes pass through the authentication middleware, which verifies the JWT before granting access to the requested resource.

18. Folder Structure

```
Tripvault/
|-- server/
|   |-- models/
|   |   |-- User.js
|   |-- routes/
|   |   |-- authRoutes.js
|   |-- middleware/
|   |   |-- authMiddleware.js
|   |-- controllers/
|   |   |-- authController.js
|   |-- config/
|   |   |-- db.js
|   |-- .env
|   |-- index.js
|
|-- client/
    |-- src/
        |-- components/
        |-- pages/
        |-- App.jsx
```

Folder Explanation

- server/models/ — Mongoose schema definitions, including the User model.
- server/routes/ — Express route definitions mapping HTTP endpoints to controller functions.
- server/middleware/ — Custom middleware, including JWT verification for protected routes.
- server/controllers/ — Business logic for registration, login, and profile retrieval.
- server/config/ — Configuration files, including the MongoDB connection setup.
- server/.env — Environment variables such as database URI and JWT secret (excluded from version control).
- server/index.js — Application entry point that initialises Express and connects middleware and routes.
- client/src/components/ — Reusable UI components such as form inputs and buttons.
- client/src/pages/ — Page-level React components, including Login, Register, and Dashboard.
- client/src/App.jsx — Root React component defining application routing.

19. Implementation

Project Setup

The project was initialised as two independent packages — client and server — within a single repository, enabling independent dependency management and deployment.

```
mkdir Tripvault && cd Tripvault
mkdir server client
cd client && npm create vite@latest . -- --template react
cd ../server && npm init -y
```

Installing Packages

```
# server dependencies
npm install express mongoose bcryptjs jsonwebtoken dotenv cors
npm install --save-dev nodemon

# client dependencies
npm install axios react-router-dom
```

Connecting MongoDB

A dedicated configuration module establishes the connection to MongoDB Atlas using Mongoose, reading the connection string from environment variables.

```
// config/db.js
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log('MongoDB Atlas connected successfully');
  } catch (error) {
    console.error('MongoDB connection failed:', error.message);
    process.exit(1);
  }
};

module.exports = connectDB;
```

Creating the User Model

```
// models/User.js
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
```

```
  password: { type: String, required: true },
  createdAt: { type: Date, default: Date.now },
});

module.exports = mongoose.model('User', userSchema);
```

Register API

```
// controllers/authController.js
exports.registerUser = async (req, res) => {
  const { name, email, password } = req.body;
  const existingUser = await User.findOne({ email });
  if (existingUser) return res.status(400).json({ message: 'User already exists' });

  const hashedPassword = await bcrypt.hash(password, 10);
  const user = await User.create({ name, email, password: hashedPassword });

  res.status(201).json({ message: 'User registered successfully', userId: user._id });
};
```

Login API

```
exports.loginUser = async (req, res) => {
  const { email, password } = req.body;
  const user = await User.findOne({ email });
  if (!user) return res.status(400).json({ message: 'Invalid credentials' });

  const isMatch = await bcrypt.compare(password, user.password);
  if (!isMatch) return res.status(400).json({ message: 'Invalid credentials' });

  const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET, { expiresIn: '7d' });
  res.status(200).json({ message: 'Login successful', token });
};
```

JWT and Middleware

```
// middleware/authMiddleware.js
const jwt = require('jsonwebtoken');

const protect = (req, res, next) => {
  const authHeader = req.headers.authorization;
  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ message: 'Not authorized, no token' });
  }
  try {
    const token = authHeader.split(' ')[1];
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.userId = decoded.id;
    next();
  } catch (err) {
```

```
    res.status(401).json({ message: 'Not authorized, token invalid' });
  }
};

module.exports = protect;
```

Testing APIs

All endpoints were tested using Postman, verifying correct status codes, response payloads, and error handling for invalid inputs, duplicate emails, and expired or missing tokens.

GitHub Repository

The complete Week 1 source code, including both client and server directories, was committed and pushed to a GitHub repository, with a structured commit history documenting each stage of development.

20. Algorithms

Registration Algorithm

10. Start.
11. Receive name, email, and password from the client.
12. Validate that all required fields are present and correctly formatted.
13. Query the database to check whether the email already exists.
14. If the email exists, return an error response and stop.
15. Otherwise, hash the password using bcrypt.js.
16. Create a new user document with the hashed password and save it to MongoDB.
17. Return a success response with the new user's ID.
18. End.

Login Algorithm

19. Start.
20. Receive email and password from the client.
21. Query the database for a user document matching the given email.
22. If no matching user is found, return an authentication error and stop.
23. Compare the submitted password with the stored hash using bcrypt.compare().
24. If the comparison fails, return an authentication error and stop.
25. If it succeeds, generate a signed JWT containing the user's ID and an expiry time.
26. Return the token to the client.
27. End.

Authentication (Protected Route) Algorithm

28. Start.
29. Intercept the incoming request at the middleware layer.
30. Extract the Authorization header and check for a Bearer token.
31. If no token is present, return a 401 Unauthorized response and stop.
32. Verify the token's signature and expiry using the JWT secret.
33. If verification fails, return a 401 Unauthorized response and stop.
34. If verification succeeds, attach the decoded user ID to the request object.
35. Pass control to the next handler (the protected route controller).
36. End.

21. Flowcharts

Registration Flowchart

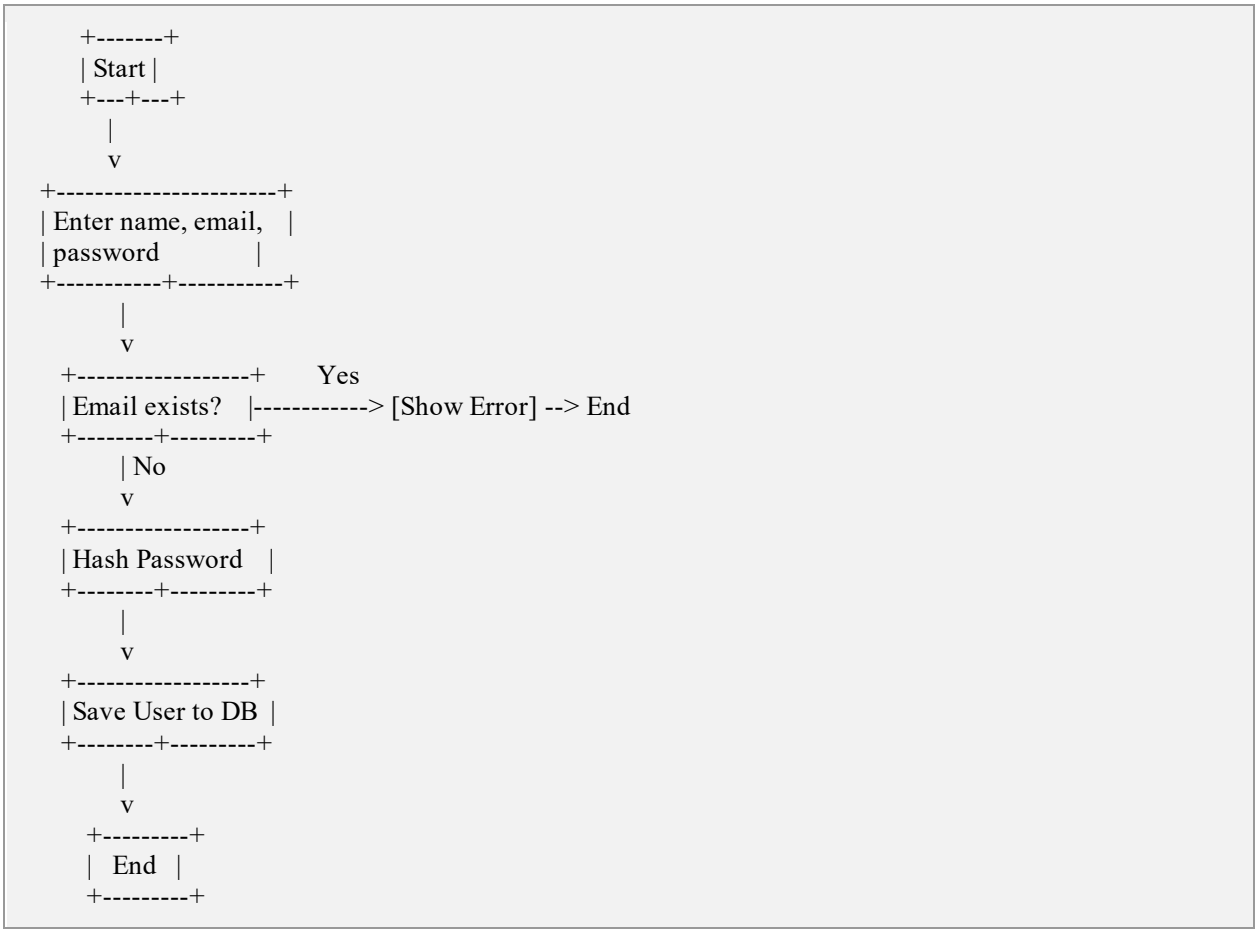
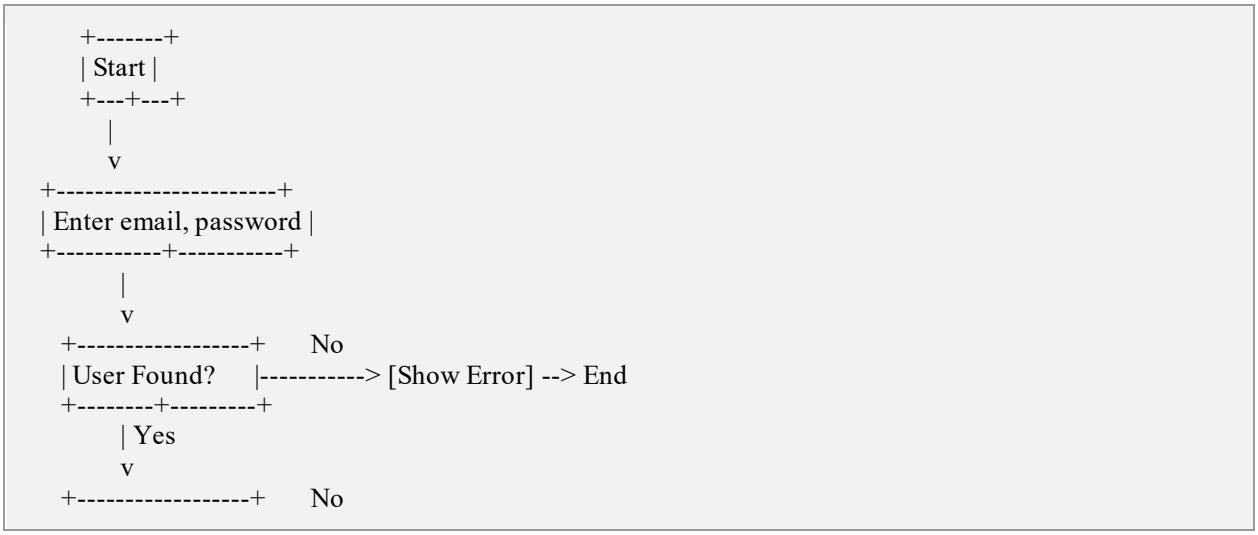


Figure 12: Registration Flowchart

Login Flowchart



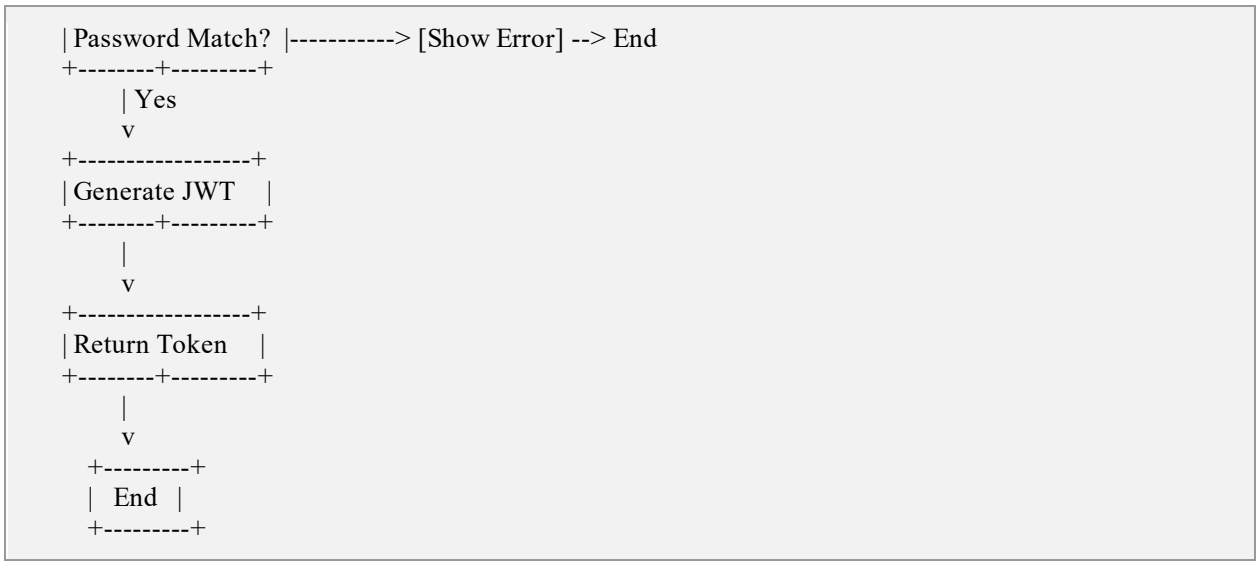


Figure 13: Login Flowchart

Authentication (Protected Route) Flowchart

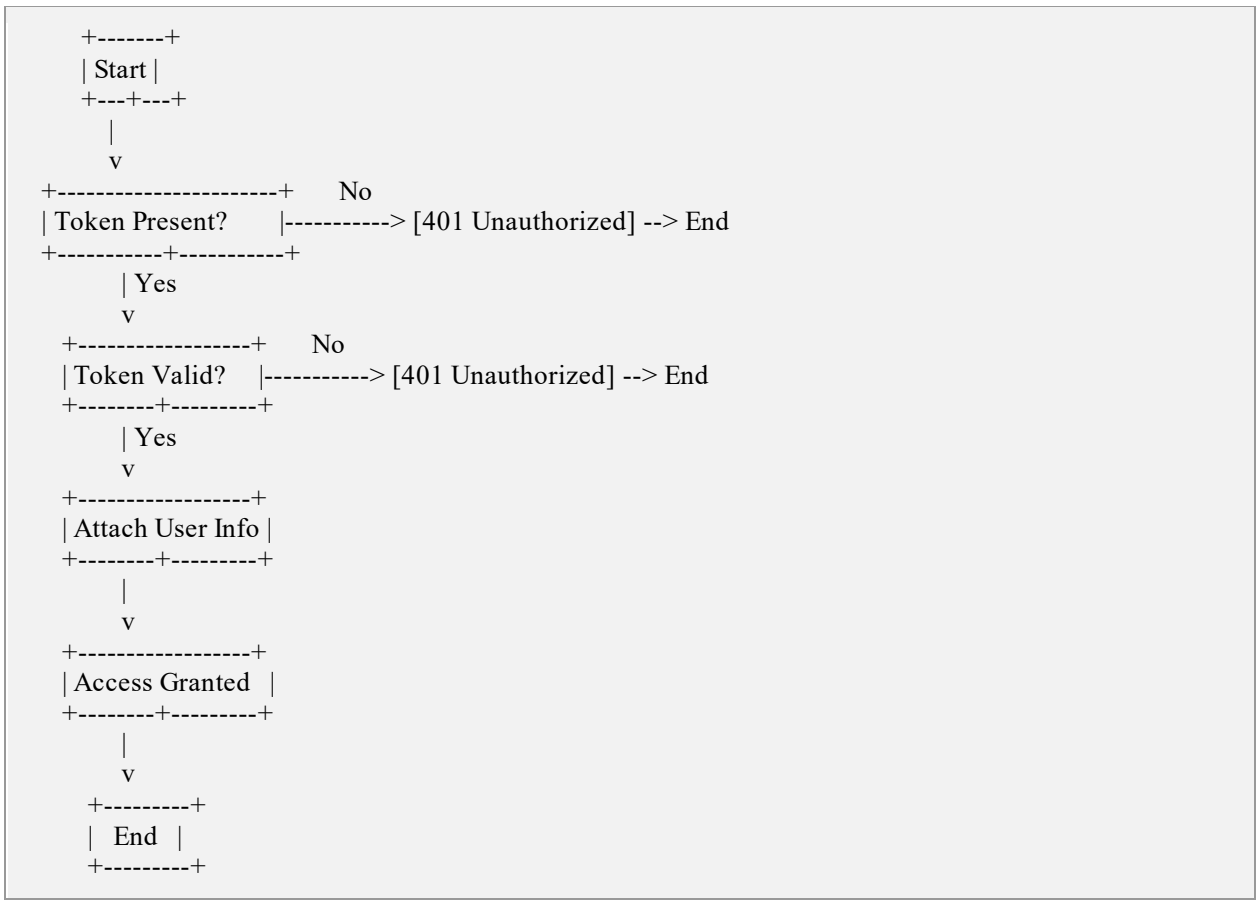


Figure 14: Authentication (Protected Route) Flowchart

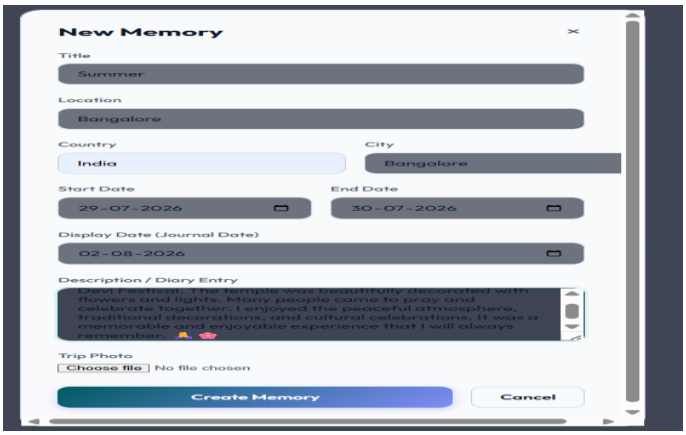


Figure 20: Tripvault Frontend — Add Memory

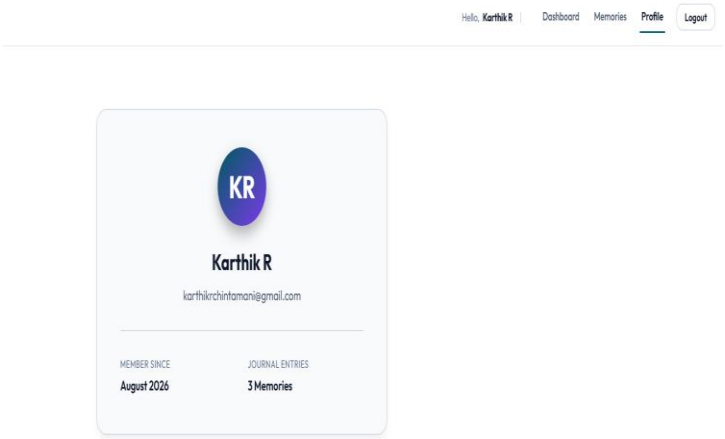


Figure 21: Tripvault Frontend — Profile Page

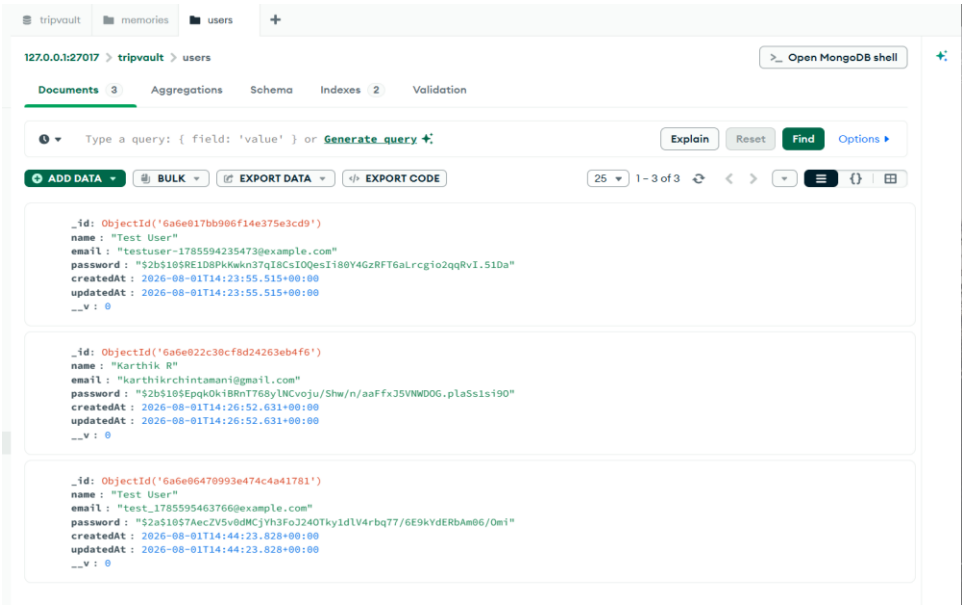


Figure 22: MongoDB Atlas — Document View

22. Results

By the end of Week 1, Tripvault has a fully functioning authentication system. New users can successfully register through the React frontend, with their credentials securely hashed and persisted in MongoDB Atlas. Registered users can log in and receive a valid JWT, which is successfully used to access a protected route that returns the authenticated user's profile. API testing in Postman confirmed correct status codes and error handling for edge cases such as duplicate registrations, incorrect passwords, and missing or invalid tokens. The codebase has been committed to GitHub with a clear, modular folder structure ready to support future feature development.

23. Advantages

- Secure-by-design authentication using industry-standard hashing (bcrypt.js) and token-based sessions (JWT).
- Modular backend architecture that cleanly separates models, routes, controllers, and middleware.
- Modern, fast frontend development experience enabled by Vite.
- Cloud-hosted database eliminates the need for local database administration.
- Stateless authentication supports easy horizontal scaling of the backend.
- Clear documentation and folder structure ease onboarding for future contributors.

24. Future Enhancements

- Travel Timeline — a chronological visualisation of a user's trips.
- Interactive World Map — highlighting visited locations geographically.
- Visited Countries Tracker — automatic tallying of countries and regions explored.
- Photo Gallery — organised storage and display of trip photographs.
- Weather API Integration — historical or forecast weather data for trip destinations.
- AI Travel Recommendation — personalised destination suggestions based on user history.
- Expense Tracker — logging and categorising trip-related expenses.
- Google Maps Integration — route mapping and location search.
- Travel Analytics Dashboard — statistics such as distance travelled and trip frequency.
- Offline Mode — local caching for access without an active internet connection.

25. Conclusion

Week 1 of the Tripvault Virtual Internship Program has successfully delivered a secure, well-structured foundation for the application. The project's technology stack — React.js (Vite), Node.js, Express.js, MongoDB Atlas, JWT, and bcrypt.js — has been integrated into a working authentication system that satisfies the functional and non-functional requirements defined at the outset. With registration, login, password hashing, JWT issuance, and protected-route access all implemented and tested, Tripvault is well-positioned to expand into its broader feature set in subsequent milestones, including trip logging, media management, and travel analytics.

26. References

- React Documentation — https://react.dev/
- Node.js Documentation — https://nodejs.org/en/docs/
- Express.js Documentation — https://expressjs.com/
- MongoDB Documentation — https://www.mongodb.com/docs/
- JWT (JSON Web Token) Documentation — https://jwt.io/introduction
- GitHub Documentation — https://docs.github.com/

27. Appendix

Installation Commands

```
npm install
npm run dev
```

Environment Variables (.env)

```
PORT=5000
MONGO_URI=your_mongodb_atlas_connection_string
JWT_SECRET=your_jwt_secret_key
```

Sample JSON Request (Register)

```
{
 "name": "Asha Rao",
 "email": "asha.rao@example.com",
 "password": "SecurePass123"
}
```

Sample JSON Response (Register)

```
{
 "message": "User registered successfully",
 "userId": "64f1c2a9e1b2c3a4d5e6f7a8"
}
```

Sample JSON Request (Login)

```
{
 "email": "asha.rao@example.com",
 "password": "SecurePass123"
}
```

Sample JSON Response (Login)

```
{
 "message": "Login successful",
 "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```